ASE CHAPTER 3

T2 Descriptive & Process Exercises Pack

Section 1: Core Textbook Descriptive Exercises (18 Items)

Exercise 3.1. Significance of Requirement Engineering & Problems in Requirements Formulation

Discuss the significance and use of requirement engineering. What are the problems in the formulation of requirements?

Significance and Use of Requirement Engineering: Requirement engineering (RE) is the disciplined process of discovering, analyzing, documenting, and validating the services a system must provide and the operational constraints under which it must function. Its primary significance lies in:

1. **Cost Containment:** Finding misunderstandings while requirements are being clarified can avoid carrying them into design, implementation, and later correction. No fixed cost multiplier is asserted here.
2. **Contractual Foundation:** The resulting specification serves as a **formal agreement and contract between customer and developer**, minimizing divergence between expectations and the delivered product.
3. **Architectural Baseline:** Requirements drive software architectural design, effort and cost estimation, project scheduling, verification planning, and acceptance testing.

Problems in the Formulation of Requirements:

1. **Requirements are rarely well-defined upfront:** Customers typically possess a vague concept of what they need and cannot articulate precise operational rules.
2. **Requirements volatility:** System requirements constantly change over time due to business environment dynamics, regulatory updates, and evolving user perspectives.
3. **Communication barrier:** A severe semantic gap exists between domain users (who speak organizational jargon) and software engineers (who think in data models, algorithms, and architectures).
4. **Natural language ambiguity:** Requirements written in natural language (e.g., English) inherently suffer from ambiguity, contradictions, and multiple valid interpretations.
5. **Omission of essential constraints:** Users routinely take critical implicit operational assumptions for granted, leaving vital non-functional constraints unstated until system failure occurs.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1, pp. 40–42; companion Slide 134.

Exercise 3.3. Crucial Process Steps of Requirement Engineering

What are crucial process steps of requirement engineering? Discuss with the help of a process flow description.

Crucial Process Steps of Requirements Engineering: The requirements engineering process takes the initial Problem Statement as input, produces the Software Requirements Specification (SRS) as output, and consists of four crucial, tightly coupled iterative process steps:

1. **Requirements Elicitation:** Proactively uncovers, discovers, and gathers user expectations, operational realities, and system constraints through direct stakeholder interaction (interviews, FAST workshops, QFD, and use cases).
2. **Requirements Analysis:** Refines, models, and scrutinizes elicited requirements to resolve conflicts, remove ambiguities, and establish system boundaries (DFD leveling, Data Dictionary, ER modeling).
3. **Requirements Documentation:** Formalizes analyzed requirements into a standardized, clear, and unambiguous technical contract—the **Software Requirements Specification (SRS)** conforming to IEEE Std 830.
4. **Requirements Review (Validation):** Rigorously inspects the draft SRS through formal peer walkthroughs and quality checklists to detect defects before implementation, feeding iterative rework loops back to Documentation and Elicitation.

The Role of Feasibility Study: *Feasibility Study* is an early, pre-project evaluation filter that precedes full requirements engineering. It evaluates technical, economic, and operational viability to decide whether to proceed to RE, re-scope constraints, or terminate the project before capital is committed.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1.1, pp. 41–42; companion Slide 3 & Slide 134.

ملاحظة أكاديمية هامة (Academic Note)

فخ الامتحان (Exam Trap): إذا طُلب رسم أو شرح خطوات RE حسب مخطط د. علي فهيم المعتمد (Figure 3.1 والسلايد 3)، فالخطوات الأربع الأساسية هي: الاستخلاص (Elicitation) ← التحليل (Analysis) ← التوثيق (Documentation/SRS) ← المراجعة (Review/Validation) مع حلقتي تغذية راجعة (Rework Loops). أما دراسة الجدوى (Feasibility Study) فهي بوابة تقييم سابقة لتحديد جدوى الدخول في هندسة المتطلبات من الأساس وليست خطوة خامسة داخل حلقة التوصيف.

Exercise 3.4. Present State of Practice in Requirement Engineering & Improvement Steps

Discuss the present state of practices in requirement engineering. Suggest few steps to improve the present state of practice.

Present State of Practice in Requirement Engineering: Recurring difficulties in requirements practice include:

1. **Requirements are difficult to uncover:** Stakeholders may not initially know or clearly express all their needs.
2. **Requirements change:** New information and changing circumstances can alter requirements.

3. **There is no single best technique:** CASE tools can help, but they do not replace sound requirements-engineering methods and processes.
4. **Project schedules may be too tight:** Insufficient time can leave requirements analysis incomplete.
5. **Communication barriers arise:** Customers and developers may use different vocabularies and have different backgrounds.
6. **Software may be market-driven:** Products may need to serve anonymous or changing customers and keep evolving through upgrades.
7. **Resources may be limited:** A project may not have enough resources to satisfy every customer request, so requirements must be prioritized.

Practical improvements (synthesis):

- Involve representative stakeholders and use suitable elicitation techniques; clarify and review requirements iteratively.
- Prioritize requests against available time and resources, and allow for change, especially in market-driven products.
- Reserve enough time for analysis and document agreed requirements clearly in an SRS.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1.2, pp. 42–43; companion Slides 4–5.

ملاحظة أكاديمية هامة (Academic Note)

الخلاصة: بعض التحديات تتعلق بفهم الاحتياجات والتواصل، وأخرى بالوقت والسوق والموارد؛ لذلك يلزم إشراك المعنيين وتوضيح المتطلبات ومراجعتها وترتيب الأولويات.

Exercise 3.5. Importance of Requirements & Major Requirement Categories

Explain the importance of requirements. How many types of requirements are possible and why?

Importance of Requirements: Requirements constitute the absolute foundation of the entire software engineering lifecycle. A flawed requirement inevitably leads to an incorrect architectural design, defective implementation, and invalid verification tests. A precise requirements specification enables accurate resource, cost, and schedule estimations, forms the baseline for project validation and customer sign-off, and prevents costly late-stage redesign.

There is no single count across all requirement classifications. The principal distinctions are separate axes:

1. **Functional and Non-Functional Requirements:** This is the main distinction between what services the system provides and constraints on those services or the system.
2. **User and System Requirements:** This is a separate distinction by level of detail and intended reader; user requirements are higher-level, while system requirements are more detailed.
3. **Interface Requirements:** These specify interactions with external systems, hardware, users, and communications interfaces; they are a category of requirements, not another name for user/system requirements.
4. **Known, Undreamed, and Unknown Requirements:** Requirements can also be classified by how well stakeholders can anticipate or recognize them. This is a different classification axis.

Thus, if “types” refers to the principal functional classification, answer **two: functional and non-functional**. The other pairs above classify requirements from different viewpoints.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1–§3.2.2, pp. 45–47; companion Slides 6–9 & Slide 134.

ملاحظة أكاديمية هامة (Academic Note)

التمييز الأساسي: التصنيف الوظيفي/non-functional يختلف عن تصنيف user/system؛ الأول يصف طبيعة المتطلب، والثاني يصف مستوى التفصيل والجمهور المقصود.

Exercise 3.6. Systematic Steps of Requirements Engineering & Essentiality

Describe the various steps of requirements engineering. Is it essential to follow these steps?

Systematic Steps of Requirements Engineering: Requirements engineering comprises four tightly coupled, iterative core activities:

1. **Requirements Elicitation:** Discovering stakeholder needs through suitable techniques such as interviews, FAST, QFD, and use cases.
2. **Requirements Analysis & Modeling:** Refining elicited statements, resolving contradictory demands, and building formal models (Data Flow Diagrams, Data Dictionaries, Entity-Relationship Diagrams) to define functional scope.
3. **Requirements Documentation (SRS):** Authoring the formal, unambiguous baseline document conforming to IEEE Std 830-1998, capturing functional, interface, performance, and operational constraints as a black-box contract.
4. **Requirements Review & Validation:** Rigorously inspecting the draft SRS through structured peer walkthroughs and 7-point quality checklists (redundancy, completeness, ambiguity, consistency) to eliminate defects before design and coding.

These activities need not form a rigid one-way sequence. Requirements review is continuous and integrated with elicitation, analysis, and documentation.

Is It Essential to Follow These Steps? The activities are all important, but they can be revisited as requirements change. Their purpose is to improve understanding and the SRS; they do not have to follow a rigid sequence, and omissions can have different effects.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1.1, pp. 41–42; companion Slide 3 & Slide 134.

ملاحظة أكاديمية هامة (Academic Note)

مهم: احفظ الأنشطة الأربع، لكن تذكّر أن مراجعة المتطلبات عملية مستمرة وليست مرحلة جامدة لا تتكرر.

Exercise 3.7. Requirements Elicitation Definition & Two Primary Techniques

What do you understand with the term "requirements elicitation"? Discuss any two techniques in detail.

Definition of Requirements Elicitation: Requirements elicitation is the practice of discovering, collecting, and uncovering the needs, desires, operational realities, and system constraints from stakeholders

through proactive collaborative inquiry. It is an active exploratory process rather than passive collection, because stakeholders rarely know their complete requirements in advance.

Technique 1: Structured Interviews:

- **Mechanism:** The analyst meets face-to-face with individual stakeholders (users, managers, domain experts) using open-ended questions (to uncover broad goals) or closed-ended questions (to confirm specific facts).
- **Advantages:** Builds personal rapport, yields deep contextual insights, reveals informal workarounds, and clarifies ambiguous operations.
- **Limitations:** Extremely time-consuming; interviewees may omit crucial tacit knowledge; statements can be subjective and contradictory across different organizational levels.

Technique 2: Facilitated Application Specification Technique (FAST):

- **Mechanism:** A joint collaborative workshop bringing developers and representative users together in an intensive neutral setting facilitated by an objective coordinator. Participants create individual candidate lists of objects, services, and constraints, merge and discuss them on wall charts, eliminate duplicates, prioritize items, and draft preliminary specifications.
- **Advantages:** Breaks organizational communication barriers, accelerates consensus, eliminates cross-departmental blame, and produces agreed-upon preliminary requirements within days.
- **Limitations:** Requires strict facilitator control; high organizational cost to gather all key stakeholders; can be dominated by aggressive personalities if not managed properly.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.1 & §3.4.3, pp. 49–53; companion Slides 16–26 & Slide 134.

ملاحظة أكاديمية هامة (Academic Note)

مقارنة امتحانية سريعة: المقابلات (Interviews) تركز على الرؤية الفردية المتعمقة لكل مستخدم، بينما ورش العمل المشتركة (FAST) تركز على بناء إجماع جماعي سريع بين المستخدمين والمطورين في جلسة واحدة.

Exercise 3.8. Inventory of Elicitation Techniques & The Most Popular Choice

List out requirements elicitation techniques. Which one is most popular and why?

Inventory of Requirements Elicitation Techniques:

1. **Interviews (Structured, Unstructured, and Semi-Structured)**
2. **Brainstorming sessions**
3. **Facilitated Application Specification Technique (FAST)**
4. **Quality Function Deployment (QFD)**
5. **Use Case Approach**
6. **Prototyping**

Most popular: Interviews. They let the analyst ask questions directly, clarify answers, and learn about users' work and expectations. No single technique is sufficient for every situation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.1, pp. 49–50; companion Slide 134.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
> **تنبيه:** لا تخلط FAST مع JAD؛ أجب باسم التقنية كما ورد في المحاضرة والكتاب.

Exercise 3.10. Quality Function Deployment (QFD) & Requirement Value Factors

Discuss quality function deployment technique of requirements elicitation. Why an importance or value factor is associated with every requirement?

Quality Function Deployment (QFD) Technique: QFD helps translate customer requirements into technical requirements. Customer requirements are grouped as normal, expected, or exciting, and each requirement is assigned an importance value.

Three Core Requirement Types in QFD:

1. **Normal Requirements:** The customer states these directly; satisfying them meets an expressed need.
2. **Expected Requirements:** The customer may not state these because they are assumed; their absence can cause dissatisfaction.
3. **Exciting Requirements:** These go beyond stated expectations and can increase satisfaction when present.

Why an Importance or Value Factor Is Associated with Every Requirement: An importance scale helps compare requirements and set priorities. The scale ranges from **1 (unrealistic; further exploration needed)** to **5 (very important)**. Requirements may then be considered achievable, deferred with a reason, or impossible and dropped; achievable requirements are implemented according to their assigned priority.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.4, pp. 52–54; companion Slide 135.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
> **فخ د. علي فاهم (Exam Trap):** تذكر دائماً تصنيف QFD الثلاثي:
>
> 1. Normal: يُطلب صراحة.
> 2. Expected: غالباً لا يُطلب صراحةً؛ غيابه يسبب عدم رضا العميل.
> 3. Exciting: ميزة مبتكرة مبهرة غير متوقعة (غيابها لا يضر ووجودها يبهر العميل).

Exercise 3.11. Use Case Approach to Elicitation & Operational Guidelines

Explain the use case approach of requirements elicitation. What are use-case guidelines?

The Use Case Approach to Requirements Elicitation: Introduced by Ivar Jacobson, the use case approach is a scenario-based elicitation and behavioral modeling technique that captures how external actors interact with the system to achieve a specific measurable goal.

- An **Actor** represents an external entity (human role, external hardware device, or outside software application) that interacts with the system.

- A **Use Case** represents a complete sequence of transactions initiated by an actor that yields an observable result of value.

Core Use-Case Authoring Guidelines:

1. **Identify System Boundaries:** Explicitly define what is internal to the software system and what remains external in the operating environment.
2. **Identify Primary and Secondary Actors:** Determine all external roles that initiate system actions or receive outputs.
3. **Focus on User Goals, Not Internal Algorithms:** Name each use case with an active verb-noun phrase representing a user goal (e.g., Issue Library Card, Generate Inward Bill), completely omitting internal processing details.
4. **Specify the Happy Path First:** Clearly document the basic flow of events (main success scenario) before identifying variations.
5. **Document Alternative and Exception Flows:** Explicitly account for failure branches, missing permissions, invalid inputs, and recovery mechanisms.
6. **Avoid Implementation Detail:** Do not specify GUI buttons, database queries, programming languages, or internal data structures in use case descriptions.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.5, pp. 55–59; companion Slides 27–35 & Slide 135.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

قاعدة ذهبية في كتابة الـUse Cases: ابدأ دائماً بصيغة فـعـل + اسم تمثل غاية المستخدم وليس شاشات النظام؛ مثلاً اكتب Withdraw Cash ولا تكتب Enter ATM Screen and Click OK Button.

Exercise 3.12. Components of a Use Case Diagram & Operational Usage

What are components of a use case diagram? Explain their usage with the help of an example.

Components of a Use Case Diagram:

1. **System Boundary:** A rectangular box enclosing the functional scope of the application; anything inside represents system functionality; anything outside represents the environment.
2. **Actors:** Depicted as stick figures placed outside the system boundary, representing external roles interacting with the system.
3. **Use Cases:** Depicted as horizontal ellipses inside the boundary containing the name of the functional goal.
4. **Communication Associations:** Solid lines connecting actors to the use cases they initiate or participate in.
5. **Stereotype Relationships:**
   - **<<include>>:** Unconditional inclusion of a shared sub-process (mandatory dependency).
   - **<<extend>>:** Conditional insertion of optional behavior triggered only under specific extension point conditions.

Illustrative Example (Banking ATM System):

- **Actor:** Customer, Bank Consortium Server.
- **System Boundary:** Automated Teller Machine (ATM).
- **Use Cases:** Withdraw Cash, Check Balance, Transfer Funds.
- **Usage & Inclusion:** Withdraw Cash has an <<include>> relationship pointing to Authenticate User PIN (every cash withdrawal unconditionally requires PIN authentication).
- **Extension:** Dispense Overdraft Warning points to Withdraw Cash with an <<extend>> relationship, executing only if the account balance drops below the threshold.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.5, pp. 58–59; companion Slides 36–50 & Slide 135.

ملاحظة أكاديمية هامة (Academic Note)

قاعدة السهم في علاقات Use Case: في علاقة <<include>> السهم ينطلق من الحالة الأصلية باتجاه الحالة المشتركة الإلزامية (Verify PIN → Withdraw Cash). أما في <<extend>> السهم ينطلق من الحالة الإضافية الاختيارية عائداً إلى الحالة الأساسية (Withdraw Cash → Overdraft Warning).

Exercise 3.17. Degree of a Relationship in ER Modeling with Concrete Examples

What is the degree of a relationship? Give an example of each of the relationship degree.

Definition of Degree of a Relationship: The degree of a relationship is the number of entity types that participate in that relationship instance.

Relationship Degrees and Concrete Examples:

1. **Unary (Recursive) Relationship (Degree 1):**
   - **Definition:** An association between occurrences of the same single entity type.
   - **Example:** An EMPLOYEE entity where an employee manages other employees (Manages relationship linking EMPLOYEE to EMPLOYEE as Supervisor and Subordinate).
2. **Binary Relationship (Degree 2):**
   - **Definition:** An association between occurrences of two distinct entity types. This is by far the most common relationship in data modeling.
   - **Example:** STUDENT enrolls in COURSE.
3. **Ternary Relationship (Degree 3):**
   - **Definition:** An association simultaneously linking occurrences of three distinct entity types.
   - **Example:** DOCTOR prescribes MEDICATION to a PATIENT. All three entities are required to uniquely define the prescription event.
4. **N-ary Relationship (Degree $N \geq 4$):**
   - **Definition:** An association involving N distinct entity types simultaneously.
   - **Example:** In a complex university logistics system: PROFESSOR, COURSE, CLASSROOM, and TIME_SLOT combined into a single 4-way scheduling association.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.3, pp. 66–68; companion Slides 87–91 & Slide 136.

> **ملاحظة أكاديمية هامة (Academic Note)**
> **تنبيه مفاهيمي:** درجة العلاقة (Degree) تعني *عدد الكيانات المشتركة* (Unary, Binary, Ternary)، بينما التعددية (Cardinality) تعني *نسبة عدد مرات الارتباط بين السجلات* (1:1, 1:N, M:N). لا تخلط بين المفهومين!

Exercise 3.22. Short Technical Notes on DFD and Data Dictionary

Write short notes on: (i) Data flow diagram, (ii) Data dictionary.

(i) Data Flow Diagram (DFD): A Data Flow Diagram is a graphical, hierarchical modeling tool that depicts the flow of data through an information system and the transformation processes applied to that data.

- **Core Elements:**
  1. **Process (Circle/Bubble):** Transforms incoming data flows into outgoing data flows.
  2. **Data Flow (Named Arrow):** Represents data packets in motion.
  3. **Data Store (Open/Parallel Lines):** Represents data at rest (files, databases).
  4. **External Entity (Rectangle / Source & Sink):** Represents outside producers or consumers of information.
- **Hierarchical Leveling:** Begins with a high-level **Context Diagram (Level 0)** establishing the whole system boundary, which decomposes into **Level 1**, **Level 2**, and detailed primitive functional processes.

(ii) Data Dictionary (DD): A Data Dictionary is a structured, centralized repository containing formal definitions, descriptions, composition schemas, and representations of all data items, flows, and stores present in the system.

- **Significance:** Acts as the single source of truth, eliminating naming ambiguities, ensuring cross-team consistency, and verifying that every data flow in a DFD is mathematically and structurally accounted for.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1 & §3.5.2, pp. 60–63; companion Slides 82–86 & Slide 137.

> **ملاحظة أكاديمية هامة (Academic Note)**
> **التكامل الوظيفي:** الـDFD يعرض مسار وتحويل البيانات رسومياً، بينما القاموس (Data Dictionary) هو المرجع المكتوب الذي يشرح بالتفصيل التركيب الدقيق لكل سهم ومخزن يظهر في ذلك المخطط.

Exercise 3.25. Throw-Away Prototypes in Large Software Systems

Explain why, for large software systems development, is it recommended that prototypes should be "throw-away" prototypes?

Role of Throw-Away Prototypes in Large Systems: A throw-away prototype can help users and developers clarify requirements before committing to the final system. **It is built quickly for analysis and user feedback, then discarded; the final system is built separately.** This is useful when the prototype is not intended to become the product.

> A prototype intended to evolve into the final product follows a different approach: it must be developed and maintained with the quality expected of that product. Thus, "throw-away" applies to the prototype artifact, not to the knowledge gained from it.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.4, pp. 68–71; companion Slide 116 & Slide 137.*

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **تذكّر:** في النموذج الرامي للتخلص منه تُحفظ نتائج التعلّم والمتطلبات، بينما يُترك النموذج البرمجي نفسه.

Exercise 3.30. Additional Desirable Characteristics of an SRS

Are there other characteristics of an SRS (besides listed in section 3.4.2) that are desirable? List a few and describe why.

> **Common SRS Quality Characteristics:** *Correct, Unambiguous, Complete, Consistent, Ranked for Importance/Stability, Verifiable, Modifiable, Traceable.*
>
> **Additional Desirable Characteristics:**
>
> 1. **Design Independence (Implementation Freedom):**
>   - The SRS must specify **what** the system must do without prescribing **how** it should be implemented. It should avoid dictating programming languages, internal database schemas, or specific component frameworks unless explicitly mandated by external client constraints.
> 2. **Maintainability and Expandability:**
>   - The SRS should be structured modularly with clear section numbering and cross-references so that future enhancements, regulatory changes, or system extensions can be integrated with minimal disruption to existing text.
> 3. **Usability by Multiple Stakeholders (Readability):**
>   - The document must be comprehensible to diverse audiences: clients and end-users must understand it to confirm business intent, while designers, programmers, and testers need precise technical criteria for implementation and verification.
> 4. **Conciseness and Non-Redundancy:**
>   - Information should be stated precisely once. Redundant statements of the same requirement introduce inconsistency risk whenever modifications occur.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.2, pp. 73–75; companion Slide 103–105 & Slide 139.*

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة مصدرية:** إحالة نص التمرين إلى §3.4.2 لا تطابق موضوع الصفحات 56–57 في النسخة الممسوحة؛ اعتمدت خصائص الـSRS من موضعها في §3.6.2، ص.73–75.
>
> **أهم ميزة إضافية:** استقلالية التصميم (Design Independence)؛ وثيقة المتطلبات السيئة هي التي تتحول إلى كراس تصميم يملي على المبرمج أسماء الجداول ولغة البرمجة بدلاً من وصف المشكلة الحقيقية.

Exercise 3.31. Definition of SRS, Advantages of Standards, & Black-Box Concept

What is software requirements specification (SRS)? List out the advantages of SRS standards. Why is SRS known as the black box specification of a system?

Definition of Software Requirements Specification (SRS): The SRS is the official, comprehensive document that completely describes the behavior, functional capabilities, performance parameters, design constraints, and external interfaces of a software system. It establishes the formal baseline agreement between customers and developers.

Advantages of SRS Standards (e.g., IEEE Std 830):

1. **Completeness Framework:** Standard templates ensure that analysts do not forget critical aspects such as security, error handling, performance limits, and hardware interfaces.
2. **Cross-Project Consistency:** Standardized organization enables developers, testers, and reviewers to locate specific information rapidly regardless of who authored the document.
3. **Facilitated Communication:** Establishes a common professional vocabulary and unambiguous structure across diverse teams and contractors.
4. **Streamlined Quality Auditing:** Allows quality assurance teams to verify that mandatory contractual and regulatory clauses are satisfied.

Why SRS Is Known as a "Black-Box" Specification: An SRS treats the software system strictly as an opaque black box: it specifies all external stimuli (inputs), all observable responses (outputs), and operational performance constraints **without exposing or prescribing the internal implementation mechanisms, internal data structures, or algorithmic logic.**

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6, pp. 72–73; companion Slide 101–103 & Slide 139.

ملاحظة أكاديمية هامة (Academic Note)

مفهوم الصندوق الأسود (Black-Box): الـSRS يوثق ما يراه المستخدم من الخارج فقط (المدخلات والمخرجات وسلوك النظام)، ولا يتدخل بما يدور داخل المحرك البرمجي من خوارزميات أو تراكيب بيانات داخلية.

Exercise 3.32. Model of a Data Dictionary, Contents, & Advantages

State the model of a data dictionary and its contents. What are its advantages?

Model of a Data Dictionary: A data dictionary defines data items and composite structures using the following notation:

- `x = a + b` : **x consists of data elements a and b**
- `x = [a|b]` : **x consists of either data element a or b**
- `x = a` : **x consists of an optional data element a**
- `x = y{a}` : **x consists of y or more occurrences of data element a**
- `x = {a}z` : **x consists of z or fewer occurrences of data element a**
- `x = y{a}z` : **x consists of some occurrences of a, between y and z**

Contents of a Data Dictionary Entry:

1. **Name of the data item**
2. **Aliases** (other names for the item)
3. **Description / purpose**
4. **Related data items**
5. **Range of values**
6. **Data structure definition / form**

Uses of a Data Dictionary: It can be used to **create ordered lists of all data items or a subset, find a data-item name from its description, and design screens and test cases**.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.2, pp. 62–63; companion Slides 85–86 & Slide 139.

ملاحظة أكاديمية هامة (Academic Note)

ترميز القاموس: `x = a + b` للتركيب، `x = [a|b]` للاختيار، `x = a` للعنصر الاختياري، و`x = y{a}z` لتحديد التكرار بين حدّين.

Exercise 3.33. Desirable SRS Characteristics, Formal Specs Advantages, & Core Issues

List five desirable characteristics of a good SRS document. Discuss the relative advantages of formal requirement specifications. List the important issues, which an SRS must address.

Five Desirable Characteristics of a Good SRS:

1. **Correct:** Every requirement stated reflects a true capability or constraint desired by the client.
2. **Unambiguous:** Every requirement statement has exactly one interpretation.
3. **Complete:** Includes all significant requirements, responses to all inputs (valid and invalid), and full definitions.
4. **Consistent:** No two statements conflict regarding functionality, terminology, or timing.
5. **Verifiable:** There exists a finite, cost-effective process by which a person or machine can prove the software meets the requirement.

Relative Advantages of Formal Requirement Specifications:

- **Mathematical Precision:** Expressing requirements in mathematical notations (e.g., Z, VDM) completely eliminates natural language ambiguity and subjective interpretation.
- **Automated Verification:** Allows mathematical theorem provers and automated tools to verify consistency and completeness before coding.
- **Disadvantage / Trade-off:** High training barrier; extremely difficult for non-technical clients to read and validate.

Core Issues an SRS Must Address:

1. **Functionality:** What the software is to do.
2. **External Interfaces:** Interactions with people, hardware, and other software.
3. **Performance:** Speed, throughput, memory, and timing constraints.
4. **Attributes (Quality):** Reliability, availability, security, and maintainability.
5. **Design Constraints:** Operating system requirements, standards compliance, and resource limits.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.1 & §3.6.2, pp. 72–75; companion Slide 101–105 & Slide 139.

ملاحظة أكاديمية هامة (Academic Note)

موازنة المواصفات الرسمية: المواصفات الرياضية (Formal Specifications) تلغي اللبس تماماً وتتيح الفحص الآلي، لكن عيبها القاتل هو عجز العميل العادي عن قراءتها، مما يصعب توقيع العقد والمصادقة عليها.

Exercise 3.35. Organization of an SRS According to IEEE Std 830 & Key Issues

Discuss the organization of a SRS. List out some important issues of this organization.

Standard Organization of an SRS (IEEE Std 830-1998 Framework):

1. **Introduction:**
   - 1.1 Purpose (intended audience and scope of document)
   - 1.2 Scope (software identification, benefits, and objectives)
   - 1.3 Definitions, Acronyms, and Abbreviations
   - 1.4 References (prior documents, standards, manuals)
   - 1.5 Overview (organization of remaining document)
2. **Overall Description:**
   - 2.1 Product Perspective (independent product or subsystem component)
   - 2.2 Product Functions (high-level functional summary)
   - 2.3 User Characteristics (educational level, technical competence)
   - 2.4 General Constraints (regulatory rules, hardware bounds)
   - 2.5 Assumptions and Dependencies (third-party factors)
3. **Specific Requirements:**
   - Detailed, verifiable requirements organized by system mode, user class, object, or functional hierarchy. Covers:
     - External Interface Requirements (User, Hardware, Software, Communications)
     - Functional Requirements (detailed inputs, processing, outputs)
     - Performance Requirements (speed, memory, concurrent users)
     - Design Constraints (standards, platform limitations)
     - Software System Quality Attributes (Reliability, Availability, Security, Maintainability)

Important Organizational Issues:

1. **Avoiding Implementation Infiltration:** Keeping section 3 focused strictly on problem specification rather than architectural solution.
2. **Structuring for Change:** Organizing functional requirements modularly so modifications affect localized sections without creating widespread ripple effects.
3. **Maintaining Traceability:** Ensuring every specific requirement has a unique hierarchical tag (e.g., FR-01.2) to support bidirectional tracing.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.3, pp. 75–78; companion Slide 106–110 & Slide 139.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **الهيكل الثلاثي لمعيار IEEE 830:**
>
> 1. Introduction: تمهيد وتعريفات ومراجع.
> 2. Overall Description: نظرة عامة شاملة بلا تفاصيل تقنية معقدة.
> 3. Specific Requirements: المتطلبات التفصيلية القابلة للفحص والتحقق للمبرمجين والمختبرين.

Synthesis 19. Feasibility Study: Purpose, Dimensions, & Bounded Risk Management

What is the purpose and focus of a feasibility study in requirements engineering? Discuss its core dimensions, bounded risk management objectives, and the resulting deliverable.

Purpose and Focus of a Feasibility Study: A feasibility study is a short, focused assessment conducted at the earliest stage of requirements engineering to evaluate the potential impact, viability, and practicality of a proposed software project.

- **Primary Objective:** Determine whether the project concept is viable and whether development should proceed, be re-scoped, or be terminated before significant financial capital and engineering effort are expended.
- **Historical Project Figures:** A historical IBM report gave estimates of **31% of projects cancelled, 53% over budget by an average of 189%, and 94 restarts per 100 projects**. Treat these as historical estimates, not current universal rates. Early feasibility assessment examines viability, realistic costs and schedules, and major risks before further resources are committed.

Core Dimensions of Feasibility Assessment:

A feasibility study considers whether the project is technically possible, whether cost and schedule estimates are realistic, and whether the project is justified and workable for its intended users and organization.

Feasibility Study Focus Checklist:

- Is the product concept viable, and can a product matching the project vision be developed?
- Are the estimated cost and schedule realistic, and how large is the gap from the original estimates?
- Is the business model justified by the estimated costs and benefits?
- Have major project risks been identified and can they be surmounted?
- Are specifications complete and stable enough to support development?
- Can users and developers agree on a detailed user-interface prototype?
- Is the development plan complete and adequate?

Risk-management scope: At this stage, identify the major project risks and assess whether they can be surmounted; this is not a full risk-management lifecycle.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.3, pp. 47–49; companion Slides 10–15; Dr. Ali Fahim Vocabulary Heading 3.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **التأطير المنهجي لدراسة الجدوى:** دراسة الجدوى ليست كتابة متطلبات تفصيلية، بل هي دراسة استكشافية للإجابة عن سؤال استراتيجي: *هل يستحق هذا المشروع الاستثمار وهل يمكن بناؤه تقنياً واقتصادياً؟* وإلغاء المشروع غير المجدي في هذه المرحلة هو نجاح هندسي يحمي الشركة من خسائر فادحة.

Synthesis 20. The Three Interface Specification Types in Software Requirements

Explain the role of interface specifications in software requirements engineering. Detail the three primary interface types highlighted in companion Slide 9 and the external interface requirements mandated by IEEE Std 830.

Role of Interface Specifications: Software systems rarely operate in complete isolation; they interact continuously with users, physical hardware devices, external databases, and third-party software services. **Interface specification defines the exact syntactic, semantic, and communication boundaries governing these interactions.** Precise interface documentation is critical for customers, system integrators, and independent module developers.

The Three Primary Interface Types:

1. **Procedural Interfaces (Application Programming Interfaces - APIs):**
   - Define the programmatic mechanism through which external software components invoke system services.
   - **Specifications include:** Function/method signatures, input parameter types, return data types, exception handling behaviors, and preconditions/postconditions.
2. **Data Structures:**
   - Define the internal and shared organization of complex information exchanged across components or stored in shared memory areas (e.g., records, queues, linked buffers).
3. **Representation of Data:**
   - Defines the physical and encoding format of data streams exchanged across network interfaces or file systems (e.g., JSON schemas, XML schemas, IEEE floating-point standards, binary packet headers).

External Interface Classification (IEEE Std 830): In a complete SRS document, interface specifications are formally partitioned into:

- **User Interfaces (UI):** Screen layouts, input styles, menu structures, keyboard shortcuts, and accessibility rules.
- **Hardware Interfaces:** Physical device addresses, baud rates, communication buses, memory-mapped I/O, and hardware interrupt signals.
- **Software Interfaces:** Operating system services, database management system (DBMS) connections, and external third-party software libraries.
- **Communications Interfaces:** Network protocol suites (TCP/IP, HTTP/REST, WebSockets), port numbers, encryption standards (TLS), and message buffering.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1, pp. 45–46; companion Slide 9; IEEE Std 830-1998 §3.1.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

تنبيه امتحاني دقيق: ميز بين التصنيف المنهجي في سلايد 9 (APIs, Data structures, Representation of data) وبين تبويب معيار IEEE 830 (User, Hardware, Software, Communications Interfaces). كلاهما يكمل الآخر لتغطية شاملة.

Synthesis 21. Requirements Review & Validation Process: Stages, Actions, & Checklist

Describe the systematic Requirements Review Process. Detail its six sequential stages, problem resolution actions, and the 7-point review checklist.

The Requirements Review Process: Requirements validation is the final phase of requirements engineering aimed at ensuring that the SRS accurately and completely specifies stakeholder intentions. **The formal review process is a multi-disciplinary inspection where analysts, domain experts, customers, and developers evaluate the specification against established quality criteria.**

Six Sequential Stages of the Review Process:

1. **Plan Review:** Define the review scope, schedule the session, establish ground rules, and select the multi-disciplinary inspection team.
2. **Distribute SRS Documents:** Deliver the draft SRS specification and review guidelines to all participants well in advance of the meeting.
3. **Read Documents (Individual Preparation):** Reviewers individually inspect the document, annotate anomalies, formulate queries, and check conformance against standards.
4. **Organize Review (Inspection Meeting):** The team convenes in a structured session to walk through the document systematically, logging all identified defects and ambiguities.
5. **Follow-Up Actions:** The review leader coordinates and tracks the agreed follow-up actions arising from the review.
6. **Revise Document:** The requirements author updates the SRS to incorporate the agreed corrections.

Problem Resolution Actions:

- **Requirements Clarification:** Clarify vague or ambiguous phrasing.
- **Missing Information:** Re-engage stakeholders to retrieve omitted business rules.
- **Requirements Conflicts:** Convene stakeholder negotiations to resolve contradictory demands.
- **Unrealistic Requirements:** Consult stakeholders to scale back infeasible scope.
- **Security Issues:** Verify and align system requirements with institutional security standards.

The 7-Point Quality Review Checklist:

1. **Redundancy:** Is the same requirement unnecessarily stated multiple times?
2. **Completeness:** Are all functions, states, inputs, and outputs defined?
3. **Ambiguity:** Can any requirement statement be interpreted in multiple ways?
4. **Consistency:** Are there conflicting constraints or mutual contradictions?
5. **Organization:** Does the document strictly conform to the established SRS standard?
6. **Conformance:** Does the specification adhere to organizational and legal standards?
7. **Traceability:** Is each requirement uniquely tagged and bidirectionally traceable?

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.7, pp. 92–94; companion Slides 112–115.

ملاحظة أكاديمية هامة (Academic Note)

قائمة الفحص السبعة (The 7 Checkpoints): احفظ نقاط التحقق السبعة لعملية المراجعة: Redundancy (عدم التكرار)، Completeness (الشمول)، Ambiguity (الوضوح)، Consistency (التناسق)، Organization (الترتيب)، Conformance (المطابقة)، Traceability (القابلية للتتبع).